The choice between :defined and :random doesn't need to be either/or. If you're migrating an entire RSpec suite from the former to the latter, it can be difficult to make the switch all at once. As you saw in the expense tracker exercise, examples can have hidden order dependencies.

You can use metadata to make the transition to random ordering more gradual. By tagging an example group with order: :random, you can run just the examples in that group randomly:

08-metadata/13/random_order.rb

```
RSpec.describe SomeNewExampleGroup, order: :random do
  # ...
end
```

Once you've transitioned all your groups to random ordering, you can remove this metadata and then turn on randomness for the entire suite in your RSpec.configure block.

On rare occasions, you'll need even more fine-grained control over spec ordering. For example, you may want to run all your unit specs first, or run all your specs in order from fastest to slowest. In these cases, you can set the :order metadata to a custom ordering. The RSpec docs explain how to do so.[2]

Your Turn

In this chapter, you've seen how to slice and dice your specs by just about any way you can think of. You can run just the fastest examples, or the ones for a specific platform, or the ones you're focusing for a particular task. The result is that you spend less time waiting for your specs to run, and more time writing code.

We've also peeled back the curtain to show you that the magic behind this flexibility is an ordinary Ruby hash. You can easily define cross-sections of your specs to fit any situation.

Now, it's time to put this newfound experience to the test.

Exercise

When we are trying to find bottlenecks in our specs, we often look at SQL operations, one of the main sources of test slowness. It can be really useful to know which SQL statements came from which examples. With your knowledge of how metadata works, you can configure RSpec to report this information.

2. http://rspec.info/documentation/3.6/rspec-core/RSpec/Core/Configuration.html#register_ordering-instance_method